TOPOLOGY / UNKNOT RECOGNITION

Component-Gauge Cut Atlases

for Compressed Unknot Hierarchies

Polynomial exact marked-cut queries for arbitrary disconnected graph grammars, with equivariant coloring certificates

Research continuation prepared for Vladimir Reshetnikov's ProveIt project
Developed with ChatGPT
12 October 2026

Abstract

The preceding sparse-cut grammar report obtains polynomial fixed-threshold evaluation for connected pieces, but retains an exponential factor in the number of terminal-bearing components. It explicitly asks whether this factor can be removed by a structured response. We resolve that question for its edge-owned graph-grammar model.

A component-gauge atlas records terminal colors relative to one root per connected component, together with root-relative monochromatic-edge flags. Independent component color flips remain symbolic. Gluing imposes XOR equations; forgetting a component eliminates its free color through a four-state OR computation. For a piece with r terminals, the number of rows of exact cut cost d is at most

$$4(d+1)\max\{1, 8r\}^d.$$

A shared-budget argument bounds candidate pairs by a degree-k polynomial in interface size. Consequently every supplied positive-integral-capacity, fresh-copy graph grammar admits deterministic polynomial-time exact evaluation at each fixed threshold k, without a connectedness, component-count, or nesting-depth restriction. In particular this applies to the two-edge-mark query at threshold three.

We also construct an equivariant coloring DAG: one recipe works for every assignment of visible component colors, so witness extraction does not restore the eliminated exponential factor. A separate verifier checks complete atlases and symbolic positive witnesses. The delivery includes source-schema compatibility with ProveIt report 91, exhaustive and randomized finite checks, binary stress cases, and paired timings, including regressions. The graph theorem is unconditional for its stated input model; a general quasi-polynomial knot recognizer still requires faithful geometric grammar production, disc transport, and global hierarchy accounting.

Status. Mathematical graph-kernel results are proved below. The standalone software and reported tests were executed. Native geometric recognition was not executed or certified.

Contents

1 The precise research advance **3**
1.1 Continuing the actual repository checkpoint . . . 3
1.2 The announced target and the local theorem are different . . . 3
1.3 Attribution and novelty boundary . . . 4

2 Graph grammars and the exact marked query **4**
2.1 Graphs, capacities, and two edge marks . . . 4
2.2 Fresh-copy, terminal-only composition . . . 5
2.3 Connectivity metadata is independent of a coloring . . . 5

3 Component-gauge atlases **6**
3.1 A row with symbolic component colors . . . 6
3.2 Small examples and a necessary correlation . . . 7

4 A component-free polynomial row bound **7**
4.1 Terminal skeletons after normalization . . . 7
4.2 Only positive-cost components can deviate . . . 8

5 Exact gluing without enumerating root colors **9**
5.1 The quotient constraint system . . . 9
5.2 Transporting marks before forgetting . . . 10
5.3 A reusable linear gluing plan . . . 11

6 Budget-preserving composition and total complexity **12**
6.1 A degree-k pair bound . . . 12
6.2 Leaf compilation without component-color enumeration . . . 12
6.3 The precise encoded-input theorem . . . 13
6.4 Minimum-only dominance is safe, but changes the contract . . . 14

7 Complete certificates and equivariant coloring witnesses **14**
7.1 Independent completeness replay . . . 14
7.2 Why an ordinary witness DAG needs care . . . 15
7.3 Original occurrence addresses and random access . . . 16

8 Application to terminal boundary-pattern tests **16**
8.1 The geometric premises must remain explicit . . . 16
8.2 What changes in the global complexity ledger . . . 18

9 Implementation, source compatibility, and trust boundaries **18**

9.1 Delivered modules and certificate contracts . . . 18
9.2 A tested adapter to the existing report-91 input schema . . . 19
9.3 Operational limitations that matter . . . 19

10 Executed validation and paired measurements 20
10.1 Finite validation . . . 20
10.2 Binary stress cases . . . 20
10.3 Paired complete-query protocol . . . 21

11 Further research questions and proposed next steps 22
11.1 A faithful native boundary-grammar exporter . . . 22
11.2 Compile signed interfaces once . . . 22
11.3 A minimum-only certificate mode and honest hybrid dispatch . . . 22
11.4 From graph addresses to violating discs . . . 22
11.5 Dynamic repairs and honest causal dependencies . . . 23
11.6 Sharper atlas bounds and extended observations . . . 23
11.7 Higher thresholds and genuine global progress . . . 23

12 Claims ledger and conclusion 23

A Source audit and attribution 24

B Reproduction and integration 25

References 27

1 The precise research advance

1.1 Continuing the actual repository checkpoint

The maintained ProveIt synthesis already describes source parallelity handles of indices zero, one and two, full base polygons, and an inherited complementary core with at most $30T$ handles for T source tetrahedra [3]. Thus an older top-level statement that bounded cores remain entirely unimplemented should not be used to select a new research target. The distinction still requiring attention is between this supplied-cut decomposition and a complete admissible patterned hierarchy.

At the inspected revision `d5c9a97fac1803258ea675297e4a7b22b78dd79b`, the report index places the disconnected-tile, sparse-cut, and causal-persistence continuations at reports 88, 91, and 92, respectively [4]. The sparse-cut article [5] is our immediate predecessor. It proves that connected pieces have only polynomially many fixed-threshold marked boundary responses. For disconnected pieces its evaluation bound has the form

$$\operatorname{poly}(s, L, W, B, k) \exp\bigl(O(C + k \log(L + W + 2))\bigr), \tag{1}$$

where C is the maximum number of connected components meeting a rule's terminal interface. Its further-research section asks for factorized responses when C is large. The present paper removes C from the exponential term by changing the retained object, rather than by bounding C geometrically or reparsing the grammar.

The earlier lower bound of 2^C is correct for an explicit listing of terminal colorings. It is not a lower bound on every representation of their response. Our construction preserves the free component choices as variables and propagates the information that those variables act upon.

1.2 The announced target and the local theorem are different

Lackenby's March 2021 Oxford handout states the announced running time as

$$2^{O((\log n)^3)} = n^{O((\log n)^2)}, \tag{2}$$

not specifically $n^{O(\log n)}$ [1, slides 7 and 25]. The July 2026 preprint gives a hierarchy algorithm; its Section 9 bounds a restart process by $L(g+1)^L$, while treating individual operation costs as separate issues [2]. We use these as motivation, not as a black-box proof that the maintained implementation already meets (2).

Our theorem concerns one exact operation inside such a program. Its input is a supplied finite graph grammar, not a knot diagram. A polynomial graph operator is useful even when geometry production and the number of hierarchy repairs remain expensive. Conversely, an enormous represented graph is not an equally enormous knot input, and its compressed evaluation is not an end-to-end recognition benchmark.

Theorem 1.1 (Main local result, informal version)**.** *Fix $k \geq 0$. Let a finite acyclic graph grammar have explicit leaves, positive integral capacities, fresh child occurrences, edge ownership, and identifications only at retained terminals. The complete marked-cut response up to threshold k has a polynomial-size component-gauge representation computable and independently replayable in deterministic polynomial time in the encoded grammar size. This holds for arbitrary disconnected pieces and arbitrary nesting. A selected finite response admits a polynomial-size symbolic coloring witness and at most k cut-edge occurrence addresses.*

The precise model, quantitative bounds and proof appear in Sections 2–7. The phrase "complete response" here means a compact functional representation; it does not promise to print exponentially many boundary assignments.

1.3 Attribution and novelty boundary

We reuse the terminal-tree counting idea and the positive-capacity budget argument from report 91, with full proofs because their normalized versions are central. Connected-component color symmetry, XOR consistency, spanning forests, and dynamic programming over a finite algebra are elementary foundations, not claimed discoveries. Threshold-preserving graph compression also has an extensive independent literature: connectivity-c mimicking networks preserve small terminal cut values, with polynomial-time constructions studied by Chalermsook and collaborators and Liu [8, 9]. Our output is not a mimicking graph, and its exact component-relative edge flags are additional data.

The contribution developed here is the closed component-gauge atlas, its component-free size and composition bounds, and its equivariant witness representation, together with their tested implementation for the supplied grammar model. This resolves the identified predecessor question in that model. We do not assert an exhaustive literature-priority claim or a general quasi-polynomial unknot theorem.

2 Graph grammars and the exact marked query

2.1 Graphs, capacities, and two edge marks

All graphs are finite undirected multigraphs; loops and parallel edges are allowed. Each edge e has a positive integral capacity $w_e \geq 1$, encoded in binary. A piece has an ordered tuple $T = (t_0, \ldots, t_{r-1})$ of distinct terminal vertices. Vertices outside T are private.

For a coloring $x : V(G) \to \mathbb{F}_2$ set

$$q_G(x) = \sum_{e=uv} w_e[\, x(u) \neq x(v)\,], \qquad F_G(x) = \{b \in \mathbb{F}_2 : \text{some edge has both endpoints colored } b\}.$$

Thus a loop always contributes a monochromatic edge, never cut cost. We encode a flag set by the mask $[0 \in F] + 2[1 \in F]$. The masks are $0, 1, 2, 3$, and flag union is bitwise OR. Write σ_b for the permutation of masks induced by replacing every color a by $a + b$; in particular σ_1 exchanges masks 1 and 2 and fixes 0 and 3.

The ordinary truncated minimum response is

$$\mathcal{R}^k_{G,T}(\chi, F) = \min\{q_G(x) : x|_T = \chi,\ F_G(x) = F,\ q_G(x) \leq k\},$$

where the empty minimum is absent. We shall retain *exact* costs up to k, a little more information than these minima. The ordinary response is recovered by minimization. At a terminal-free root, the marked query asks whether some coloring of cost at most k has flag mask 3.

Positive integral capacities have two separate roles. They ensure that at most k graph edges are cut at threshold k, and ensure that a child costing more than k cannot become feasible in a parent by cancellation. The proof does not treat arithmetic on arbitrarily large capacities as unit cost.

2.2 Fresh-copy, terminal-only composition

A grammar is a topologically ordered list of rules. An explicit leaf lists all its vertices and weighted edges, together with its terminal tuple. A unary rule identifies child terminals according to a map onto explicitly listed quotient vertices and retains an ordered subset of those vertices as new terminals. A binary rule takes two fresh disjoint child occurrences, identifies their terminals by quotient maps, and retains such an ordered subset. Several terminals of the same child may be identified.

Every edge remains owned by one child occurrence. No rule identifies, deletes or reweights graph edges. An identification can turn an edge into a loop, but both original endpoints must then receive the same color. A private vertex is never identified or promoted into a later interface. Reusing one rule twice creates distinct private vertices and edges; a DAG is a shared *recipe*, not an aliasing instruction.

Example 2.1 (Serial powers)**.** A tile with w input and w output terminals can be serially joined by identifying output with input. Repeated squaring describes a positive power N in $O(\log(N+1))$ rules. The tile need not be connected. Different outputs may be permuted, and different recipes may be nested nonuniformly. The theorem analyzes the supplied DAG directly, without expanding the power or finding a common permutation period.

The software's leaf schema explicitly lists vertices as $[0,\dots,n-1]$. A binary count alone cannot silently request allocation of 2^{128} unlisted isolated vertices. The compatibility adapter for report 91 instead validates that report's edge/terminal/private-isolate inventory before allocating this explicit list.

2.3 Connectivity metadata is independent of a coloring

Let $C_0,\dots,C_{c-1}$ be the connected components of G meeting T, ordered by their first terminal. They are *visible*. Other components are *hidden*. Throughout, connected component means a component of the whole piece, *not* a monochromatic component or a component remaining after cut edges are deleted. Let a_i be the first terminal in C_i.

At a join, the child visible components induce connectivity relations on the quotient vertices. Taking their transitive closure gives precisely the parent components meeting those vertices. A parent component is visible if it contains a retained terminal; otherwise it becomes hidden. Old hidden components can never reconnect because none of their vertices is addressable by a later rule.

The representation also tracks expanded vertex/edge counts in binary, the number of hidden components, terminal degrees saturated at four, and whether every private vertex has degree three. If the children have counts v_A, v_B, terminal counts r_A, r_B, and Q is the quotient-vertex set, then

$$v_{A\vee B} = v_A + v_B - r_A - r_B + |Q|, \qquad e_{A\vee B} = e_A + e_B.$$

Degrees at a quotient vertex add, including both incidences of any new loop. A forgotten vertex must have degree three to retain the private-cubic flag. These metadata are checked independently; none certifies an embedding or a 3-ball.

3 Component-gauge atlases

3.1 A row with symbolic component colors

Definition 3.1 (Normalized row)**.** For a coloring x, define its visible relative terminal vector by

$$p_j = x(t_j) + x(a_i) \quad \text{when } t_j \in C_i.$$

Its ith visible tag M_i is the set of colors of monochromatic edges in C_i, measured relative to $x(a_i)$. Its hidden mask H is the actual union of monochromatic colors over all hidden components. A row is

$$z = (d, p, (M_0, \dots, M_{c-1}), H), \qquad 0 \le d \le k, \tag{3}$$

realized by some coloring of exact cost d. The *component-gauge atlas* $\mathcal{A}^k_{G,T}$ is the set of all such rows.

The root coordinate of p is zero in every visible component. "Gauge" simply denotes its still-free choice of root color. There is no exponential list of these choices in a row. We do not quotient hidden colors: their actual effect is retained in the four-valued mask H.

Lemma 3.2 (Equivariant realization)**.** *If a row (3) is realized, then for every $b = (b_0, \dots, b_{c-1}) \in \mathbb{F}_2^c$ it has a realizing coloring with $x(a_i) = b_i$. For that coloring,*

$$x(t_j) = p_j + b_i \quad (t_j \in C_i), \qquad F_G(x) = H \vee \bigvee_{i=0}^{c-1} \sigma_{b_i}(M_i). \tag{4}$$

The cost remains d and the hidden mask remains H.

Proof. Start with one realizing coloring. Independently complement precisely the visible components whose root colors disagree with b. Every edge lies entirely within a single graph component, so equality or inequality of its endpoint colors is unchanged. Relative terminal colors and relative monochromatic tags are also unchanged. Hidden components were not complemented. Equation (4) follows directly from the definition of relative color. □

This elementary independence is what permits normalization. After two components are joined their root choices may no longer be independent; the gluing algorithm must impose the resulting parity constraints before eliminating any choices.

Proposition 3.3 (Recovering the original query)**.** *Given an atlas and a single supplied terminal coloring χ, all finite values $\mathcal{R}^k_{G,T}(\chi, F)$ can be recovered by scanning the rows, without enumerating component root assignments. At a terminal-free root, the exact achievable pairs are simply (d, H) from the rows.*

Proof. The supplied coloring fixes $b_i = \chi(a_i)$. A row is compatible precisely when $\chi(t_j) + b_i = p_j$ for every terminal in C_i. For a compatible row compute (4) and minimize d at the resulting flag mask. Lemma 3.2 proves realization, and every actual coloring produced one of the scanned rows. With no terminals there are no visible components or free visible bits. □

3.2 Small examples and a necessary correlation

A single unit edge whose two endpoints are terminals has two rows at threshold at least one. The first has $d = 0$, relative terminal colors $(0, 0)$ and tag $\{0\}$. The second has $d = 1$, colors $(0, 1)$ and empty tag. Complementing the root automatically gives the other two actual endpoint colorings.

For a path $u - w - v$ with terminals (u, v), there are two distinct cost-one rows with relative boundary $(0, 1)$. They have tags $\{0\}$ and $\{1\}$, depending on the color of w. These rows cannot be merged merely because their parity vectors and costs agree. For example, attach an edge of capacity greater than one at u, and then forget all terminals. At total cost one the added edge must be monochromatic with u. The first path row then gives just one global mark, while the second gives both. Thus the tag orientation matters in a permitted gluing context, even when the final interface is empty.

Now take two disjoint edges with one terminal on each. At zero cost one row has tags $(\{0\}, \{0\})$, with two free root bits. Opposite root colors give both global marks. Identifying the two terminals forces equal root colors, so the resulting two-edge path does *not* have a zero-cost both-mark coloring. This explains why an already-aggregated global flag detached from its component variables is not a safe replacement for the component-relative tags.

Hidden components must not be deleted. One hidden edge permits mask 1 or 2 at zero cost, but not 3. Two hidden edges permit mask 3 as well. The four-state aggregate retains exactly this information even when the number of hidden components is enormous.

4 A component-free polynomial row bound

4.1 Terminal skeletons after normalization

For $r \geq 1$ put

$$m(r) = \begin{cases} 0, & r = 1, \\ 2r - 3, & r \geq 2. \end{cases}$$

Lemma 4.1 (Normalized terminal skeleton bound)**.** *In a connected positive-integral-capacity graph with $r \geq 1$ terminals, the number of normalized terminal vectors induced by colorings of cost at most h is at most*

$$S(m(r), h) = \sum_{j=0}^{\min\{m(r),h\}} \binom{m(r)}{j}.$$

At exact cost zero there is exactly one normalized vector and exactly one relative edge tag: $\{0\}$ if the component has an edge, and the empty tag otherwise.

Proof. Choose a spanning tree and take its minimal subtree containing all terminals. Suppress every nonterminal vertex of degree two. For $r \geq 2$, all leaves of the resulting tree are terminals and every remaining nonterminal vertex has degree at least three. If there are b nonterminal vertices, the degree sum yields $b \leq r - 2$, so the skeleton has at most $2r - 3$ edges. For $r = 1$ keep only that terminal.

Every skeleton edge represents a path of the original spanning tree, and these paths are edge-disjoint. If the two endpoint colors of a skeleton edge differ, its path contains at least one cut graph edge. At most h graph edges can be cut, hence at most h skeleton edges are bichromatic. Once the first terminal has been normalized to zero, their subset determines every skeleton vertex color and therefore the normalized terminal vector. Counting subsets proves the bound. At cost zero all graph

edges have equal endpoint colors; connectedness makes the component monochromatic, proving the tag assertion. □

The skeleton is a counting device, not an object the grammar evaluator must construct in the expanded graph. For $h \geq 1$ we use the convenient inequality

$$S(m, h) \leq (m + 1)^h. \tag{5}$$

Indeed, sort the chosen subset of $\{1, \ldots, m\}$ and pad to length h with zeros. This injects the subsets of size at most h into length-h words over $m + 1$ symbols.

4.2 Only positive-cost components can deviate

Let $A_d(G, T)$ denote the number of atlas rows of exact cost d. Write $r_i = |T \cap C_i|$ and $m_i = m(r_i)$. Unlike an explicit coloring table, the zero-cost state of a visible component has no root-bit multiplicity.

Theorem 4.2 (Exact-degree atlas bound)**.** *For every graph in the stated model with r terminals and every integer $d \geq 0$,*

$$A_d(G, T) \leq 4 \sum_{h=0}^{d} (8r)^h \leq 4(d + 1) \max\{1, 8r\}^d. \tag{6}$$

In the first sum use $0^0 = 1$. In particular, with $R = \max\{1, 8r\}$,

$$|\mathcal{A}^k_{G,T}| \leq 4(k + 1)^2 R^k. \tag{7}$$

These bounds do not depend on the number or size of hidden components and contain no factor exponential in the number of visible components.

Proof. Consider visible component C_i at exact local cost h_i. At $h_i = 0$ its normalized vector and tag have only one possibility by Lemma 4.1. For $h_i \geq 1$ their number is at most

$$4S(m_i, h_i) \leq 4(m_i + 1)^{h_i} \leq \big(4(m_i + 1)\big)^{h_i}.$$

Let $a_i = 4(m_i + 1)$. Fix total visible cost h. Summing over all weak compositions $\sum_i h_i = h$ bounds the number of visible descriptors by

$$\sum_{h_0 + \cdots + h_{c-1} = h} \prod_i a_i^{h_i} \leq \left(\sum_i a_i \right)^h.$$

For $h = 0$ both sides are interpreted as one, including the case of no visible components. For $h > 0$ and no visible components the count is zero. The inequality follows from the multinomial expansion: every term in the left sum occurs with coefficient at least one on the right.

Since $m(r_i) + 1 \leq 2r_i$, we have $\sum_i a_i \leq 8r$. At exact total cost d, the visible cost is some $h \in \{0, \ldots, d\}$; the hidden cost is then $d - h$. All hidden components together contribute only the single four-valued mask H. They cannot add another visible descriptor. Multiplication by four and summation over h gives the first inequality of (6). The second is immediate. Summing over $0 \leq d \leq k$ and using $R \geq 1$ gives (7). □

The proof overcounts distributions of visible and hidden costs. This is harmless: an atlas row need not store which component paid which part of d. Its single realizing coloring and independent component complementation are sufficient for all later operations.

Corollary 4.3 (Exponential savings are possible at zero cost)**.** *There are pieces with c terminals and c visible components whose explicit response has 2^c finite boundary assignments but whose atlas has one row at every threshold below their edge capacities.*

Proof. Take c disjoint edges, each with one endpoint terminal and each of capacity $k+1$. No edge can be cut at threshold k. The atlas consists of $d = 0$, all relative terminal bits zero, every visible tag $\{0\}$, and no hidden marks. Every independent assignment of the c root bits is an actual zero-cost boundary coloring. The still simpler example of c isolated terminals works at every threshold. □

This is a separation between representations of a supplied interface. It is not a lower bound against every reorganization of the graph's parse. The theorem is useful precisely because it does not require a favorable reorganization.

Proposition 4.4 (The degree in the threshold is genuinely necessary)**.** *For fixed $k \geq 1$, connected graphs can have $\Omega_k(r^k)$ distinct atlas rows at threshold k.*

Proof. Take a unit-capacity star with r vertices, every vertex terminal, and list the center first. Normalize the center to zero. Any chosen subset of at most k leaves may be colored one, giving a different relative terminal vector and cost equal to the subset size. There are $\sum_{j \leq k} \binom{r-1}{j}$ such vectors. For $r - 1 \geq k$ the degree-k lower bound follows. □

Thus removing component root choices does not remove the genuine low-cut perturbations. The proposed atlas is polynomial at fixed k, not uniformly polynomial when k grows arbitrarily with the input.

5 Exact gluing without enumerating root colors

5.1 The quotient constraint system

Consider a binary rule with child terminal maps $\pi_A : T_A \to Q$ and $\pi_B : T_B \to Q$, and retained parent terminals $U \subseteq Q$. Unary rules are the same construction with one child. Take child rows z_A, z_B of costs d_A, d_B, with $d_A + d_B \leq k$.

Introduce one variable $y_q \in \mathbb{F}_2$ for each quotient vertex $q \in Q$. For every visible component of a child, with its first terminal a_i, impose

$$y_{\pi(t_j)} + y_{\pi(a_i)} = p_j \qquad (t_j \in C_i). \tag{8}$$

This includes identifications within one child. A contradiction discards the candidate. No satisfying root assignments are enumerated.

Ignoring the labels on these equations yields the connectivity graph of child visible components after identification. Thus its connected components are the parent graph components meeting Q, even when a particular row has nonzero cut cost. Choose one temporary root in each such component D. If the equations are consistent, they determine offsets ϵ_q with

$$y_q = b_D + \epsilon_q, \qquad q \in D, \tag{9}$$

where the single bit b_D remains free. A parity disjoint-set structure computes these offsets in the producer. The independent checker traverses the signed constraint graph instead.

Lemma 5.1 (Constraint completeness)**.** *The child rows admit colorings that descend to the quotient if and only if* (8) *is consistent. For every choice of the free bits in* (9)*, the corresponding child visible root colors can be realized simultaneously.*

Proof. Any coloring of the quotient pulls back to child colorings and hence satisfies their terminal-relative equations. Conversely, a solution of the equations fixes a root color for every child visible component. Lemma 3.2 supplies child realizing colorings with exactly those root colors, without altering their cost or hidden mask. The equations ensure equality at every pair of identified terminal vertices. Private vertices are not identified, so there are no omitted gluing constraints. The child colorings therefore descend to the quotient. □

5.2 Transporting marks before forgetting

If child component C_i has first terminal a_i mapping into D, transport its tag to the temporary root of D by

$$M_i \longmapsto \sigma_{\epsilon_{\pi(a_i)}}(M_i).$$

OR these transported tags over all child components in D, giving M_D. The child hidden masks combine to a fixed mask $H_0 = H_A \vee H_B$. These old hidden components remain independent of the quotient and require no new color decision.

For a component D meeting U, take its first retained terminal u_D as the new canonical root. Its parent terminal parities and tag are

$$p'_u = \epsilon_u + \epsilon_{u_D}, \qquad M'_D = \sigma_{\epsilon_{u_D}}(M_D). \tag{10}$$

Its actual root color remains symbolic. Components D not meeting U must instead be hidden. They may contribute either M_D or $\sigma_1(M_D)$ to the actual hidden mask.

Lemma 5.2 (Four-state elimination of hidden gauges)**.** *For newly hidden components $D_1, \dots, D_q$, the set of feasible parent hidden masks is computed by*

$$\mathcal{H}_0 = \{H_0\}, \qquad \mathcal{H}_{j+1} = \{H \vee \sigma_b(M_{D_{j+1}}) : H \in \mathcal{H}_j,\ b \in \mathbb{F}_2\}.$$

Every intermediate set has at most four elements. The computation takes $O(q+1)$ operations on constant-size masks. A predecessor pointer per mask recovers one actual choice of each hidden root bit for any desired final mask.

Proof. Different connected components of the quotient have independent free bits in (9). Once a component is hidden, its only retained effect is the union of its monochromatic colors with previously accumulated flags. The displayed recurrence exhausts exactly its two possible root orientations. Induction proves equality of the feasible sets. Each mask belongs to $\{0, 1, 2, 3\}$, so storing one predecessor for each reachable mask yields the stated size and time bounds. □

The dimension q of the eliminated space can be large. The algorithm does not traverse its 2^q assignments; it traverses the four possible aggregate answers.

Theorem 5.3 (Exact atlas composition)**.** *For every cost-feasible child-row pair, solve* (8)*, perform* (10)*, and emit one parent row of cost $d_A + d_B$ for each final mask in Lemma 5.2. The union of all emitted rows, with duplicates removed, is exactly the parent atlas. The analogous unary operation is exact and retains the child cost.*

Proof. For soundness, choose a final hidden mask and use the predecessor pointers to choose the newly hidden root colors. Choose arbitrary visible parent root colors and extend them by (9). Lemma 5.1 realizes compatible child colorings. Since every parent edge is exactly one child-occurrence edge, the cost is the sum of their costs. Edges that become loops are monochromatic because their endpoints were identified consistently. Transporting each relative tag and taking OR computes the actual parent marks exactly. Thus every emitted row is realized, for every visible root assignment.

For completeness, begin with an actual parent coloring of cost at most k. Its restrictions to the children have nonnegative costs d_A, d_B adding to the parent cost, so each belongs to the corresponding truncated child atlas. Their relative data satisfy (8). The actual quotient coloring determines free bits (9); its parent relative data are exactly (10). Its hidden choices are among those considered by Lemma 5.2. Therefore its parent row is emitted from this child pair. Unary identification and forgetting use the same argument without a second summand. □

Corollary 5.4 (Contextual sufficiency)**.** *Pieces with the same ordered interface, the same interface component partition, and the same exact atlas can replace one another in every context made of the permitted operations without changing any root marked response through threshold k.*

Proof. The composition theorem uses only the listed child information and the explicit quotient maps. Apply it successively to every ancestor of each occurrence of a replaced rule. □

This is a statement about the specified response. It does not preserve an embedding, graph isomorphism, surface genus, fundamental group, or a surrounding 3-manifold. Connectivity and cubic metadata needed by a geometric caller should be compared separately when replacing graph-query pieces.

5.3 A reusable linear gluing plan

The parity-disjoint-set implementation is not the only way to evaluate a join. Its unlabelled constraint graph depends only on the child component partitions and the quotient maps, not on the particular rows.

Proposition 5.5 (Precompiled consistency and transport)**.** *For a fixed join interface, choose a spanning forest of its unlabelled constraint graph. There are fixed linear maps over $\mathbb{F}_2$ that send the child parity vectors to (i) a list of cycle-consistency checks, (ii) all quotient offsets, and (iii) all parent parities and child-to-parent root shifts. With $O(W)$ combined terminal occurrences, a conservative representation of these maps uses $O(W^2)$ bits.*

Proof. Orient the constraints arbitrarily. Relative to a forest root, the offset of any quotient vertex is the sum of edge labels along its unique forest path. Every such label is one coordinate of a child parity vector or zero. Each nonforest edge is consistent precisely when its label agrees with the difference of its endpoint offsets, equivalently when the sum around its fundamental cycle is zero. All required quantities are therefore fixed XORs of input coordinates. There are $O(W)$ coordinates and $O(W)$ outputs/checks, giving the displayed conservative matrix bound. □

This proposition identifies a concrete optimization for the measured small-width regressions. The delivered reference producer deliberately uses the simpler parity data structure; it does not implement this precompiled plan or claim timings for it.

6 Budget-preserving composition and total complexity

6.1 A degree-k pair bound

A naive Cartesian product of two polynomial tables suggests degree $2k$. Exact cost indexing avoids that loss.

Theorem 6.1 (Component-free shared-budget bound)**.** *Suppose both child interfaces have at most W terminals and put $R = \max\{1, 8W\}$. Before checking quotient consistency, the number of child-row pairs with total cost at most k is at most*

$$P_k(W) = 16\binom{k+4}{4}R^k. \tag{11}$$

Each such pair produces at most four candidate parent rows.

Proof. By Theorem 4.2, the number of pairs at costs (a, b) is at most $16(a+1)(b+1)R^{a+b}$. Restrict to $a + b \le k$ and use $R \ge 1$. The resulting sum is bounded by

$$16R^k \sum_{a+b\le k} (a+1)(b+1) = 16R^k\binom{k+4}{4}.$$

For the identity, write $a = a_1 + a_2$, $b = b_1 + b_2$, and introduce a nonnegative slack variable. Counting the five-tuples of sum k gives $\binom{k+4}{4}$. The factor four for emitted rows is Lemma 5.2; consistency and duplicate removal only reduce the count. □

The bound is intentionally conservative. Its relevant feature is degree k and the absence of 2^C. At $k = 3$ it is $O((W+1)^3)$ pairs, even if each child has W separate visible components.

The implementation groups the right child by exact cost. For each left row of cost a, it processes only populated right buckets of cost at most $k - a$. The same indexing principle is used independently by the complete checker. Both algorithms still pay the interface-processing cost for each candidate; fewer candidates do not imply that a Python implementation must be faster on every small-width case.

6.2 Leaf compilation without component-color enumeration

Lemma 6.2 (Forest-switch leaf compiler)**.** *Let an explicit leaf have v vertices and c_{all} graph components, including isolates. Its atlas up to threshold k can be constructed by considering at most*

$$\sum_{j=0}^{\min\{v-c_{\mathrm{all}},k\}} \binom{v-c_{\mathrm{all}}}{j}$$

forest-switch subsets, with polynomial bit work per subset and a four-state hidden-mask computation. No factor $2^{c_{\mathrm{all}}}$ is required.

Proof. Choose a spanning forest and give each component an arbitrary temporary root color zero. Enumerate subsets of at most k forest edges across which the color changes. Propagate colors in the forest, then evaluate all original edges, rejecting actual cut cost above k. Every low-cost coloring has such a switch subset. The temporary root color omits only independent whole-component

complements. For visible components normalize at their first terminal. For hidden components apply Lemma 5.2 to their temporary-root tags. This restores all possible actual hidden masks without enumerating root assignments. Conversely each retained row is obtained from an actual forest coloring and valid component complements. □

For an empty leaf, the empty forest yields the one row $(0, \varnothing, \varnothing, 0)$. A component with no edges has no switches or edge tags, but its terminal root remains a valid free variable if visible.

6.3 The precise encoded-input theorem

Let s be the number of grammar rules, W the largest terminal count of a rule, and L the total number of explicitly listed leaf vertices and edges, with a harmless additive one. Let B bound the bit lengths needed for capacities, indices, and expanded graph-count metadata. The listed quotient maps and terminal tuples are part of the input, so their lengths are charged explicitly.

Theorem 6.3 (Unrestricted disconnected grammar evaluation)**.** *The complete atlases of all rules, their compositional metadata, and realizing backpointers are computable in deterministic bit time and space of the form*

$$\mathrm{poly}(s, L, W, B, k)\exp\big(O(k\log(L+W+2))\big). \tag{12}$$

An independently implemented complete replay satisfies a bound of the same form. In particular, every fixed k gives polynomial time and output size in the encoded grammar size. No bound on connectedness, number of terminal-bearing components, or grammar depth is needed.

Proof. Process rules in topological order. At leaves, Lemma 6.2 gives $(L+1)^{O(k)}$ candidates, each evaluated in polynomial time in the explicit data and capacity bit lengths. At a unary rule, scan the child rows, solve the parity constraints, renormalize and eliminate hidden components. At a binary rule, Theorem 6.1 bounds the cost-feasible candidates by $(k+1)^{O(1)}(W+1)^k$ up to fixed constants. Each signed-constraint graph has $O(W)$ vertices and edges. Its solution, tag transport, hidden-mask DP and metadata computation have polynomial bit cost.

Theorem 4.2 bounds every deduplicated row table. A row stores $O(W)$ bits of terminal parity and component tags, an exact cost of $O(\log(k+1))$ bits, and a two-bit hidden mask. A realizing internal backpointer names at most two child rows; a leaf backpointer lists at most k cut edges. Deterministic sorting and comparison-based dictionaries suffice to deduplicate; any logarithmic sorting factors are absorbed into the polynomial factor of (12). The reference implementation uses Python sets and dictionaries, but the stated complexity classification does not require a unit-cost arbitrary-integer or adversarial-hashing assumption.

The complete checker's leaf algorithm, proved in Section 7, enumerates subsets of at most k explicit edges. This has $(L+1)^{O(k)}$ candidates. Its internal rules independently solve the same mathematical constraints by graph traversal and use shared cost indexing. Therefore it has the same form of bound. Summing across s rules establishes the theorem. □

The expanded vertex and edge counts are at most 2^s times the largest explicit leaf size: each binary rule adds two fresh child counts before identifications can reduce the vertex count. Hence these counts require only $O(s+\log(L+1))$ bits. No loop in the producer, checker or witness compiler iterates once per expanded occurrence. A large capacity still must be read and compared at its actual bit length. The numeric parameter k is not charged merely by $\log k$; it appears in the exponent and in budget enumeration.

Corollary 6.4 (Local quasi-polynomial regime)**.** *Suppose s, L, W, B are polynomially bounded in $n \geq 2$. If $k = O(\log n)$, the supplied graph query runs in $n^{O(\log n)}$ time. More generally $k = O((\log n)^a)$ gives $2^{O((\log n)^{a+1})}$ time for fixed $a \geq 0$. At the topological threshold $k = 3$, the graph-query time is polynomial.*

Proof. Substitute $\log(L + W + 2) = O(\log n)$ into (12). The remaining polynomial factors contribute at most polynomial time in n and k. □

This local $n^{O(\log n)}$ statement should not be substituted for Lackenby's different announced global exponent (2). They quantify different inputs and different operations.

6.4 Minimum-only dominance is safe, but changes the contract

Proposition 6.5 (Descriptor dominance)**.** *For purposes of the ordinary minimum response in every permitted context, among rows with the same $(p, (M_i), H)$ it is sufficient to retain the least cost. Performing this reduction after every grammar operation preserves all truncated marked minimum responses.*

Proof. A higher-cost row with the same descriptor has exactly the same visible root choices, terminal colors and flags under (4). Replacing it by the least-cost row preserves compatibility at every quotient and cannot increase any eventual total cost. Conversely the retained row is realized. Applying this replacement successively through a grammar context preserves the minimum at every response key. □

The delivered version deliberately certifies *all exact achievable costs* through k. It does not enable this minimum-only pruning. A future implementation must use a separately identified certificate mode: silently deleting exact-cost rows from the current schema is an incomplete certificate, even when minimum answers happen to remain correct.

7 Complete certificates and equivariant coloring witnesses

7.1 Independent completeness replay

A positive coloring witness establishes existence, not absence and not minimality. A terminal no-cut conclusion therefore uses full atlas replay. The packet includes the source graph grammar, all per-rule metadata, the sorted row tables, and one realizing origin per row. The caller may supply an expected grammar, which the checker compares by typed structural equality. This binds the statement to that graph recipe, but is not a geometric authentication of the recipe.

Lemma 7.1 (Independent leaf completeness by edge cut sets)**.** *Enumerate every edge subset D with $|D| \leq k$ and $\sum_{e \in D} w_e \leq k$. Impose the equations*

$$x(u) + x(v) = [e \in D] \qquad (e = uv)$$

on all *leaf edges. For a consistent system, normalize visible components and eliminate hidden root choices by the four-state flag computation. The resulting rows are exactly the leaf atlas.*

Proof. Consistency means that there is a coloring whose cut-edge set is exactly D. Solving the equations by graph traversal leaves one free bit per original graph component. Their normalization and hidden-mask elimination are therefore exact. Every low-cost coloring has at most k cut edges and its actual set D is enumerated. Inconsistent sets, including a proposed cut loop, cannot contribute. This proves both directions without using the producer's spanning-forest-switch enumeration. □

At internal rules the checker recomputes every cost-feasible combination using breadth-first parity propagation, recomputes the metadata, and demands equality with the supplied row set. It separately verifies each origin. Thus omitted rows, altered costs, forged tags or metadata, inconsistent child references, and wrong source bindings are rejected. Budget exhaustion is inconclusive, never a no-cut certificate.

7.2 Why an ordinary witness DAG needs care

An actual coloring request includes one root bit per visible component. Merely memoizing by a row while assuming a single fixed coloring can be wrong: two occurrences of the same row may require different component flips. On the other hand, memoizing by every possible root-bit tuple would restore the exponential factor that the atlas removed.

The appropriate witness is a *family* of colorings with the visible roots as formal inputs. A Boolean expression is either a constant b or $X_i + b$, where X_i is one of the parent's visible root variables and addition is mod two. These expressions are closed under the component transports arising in a join; arbitrary Boolean formulas are unnecessary.

Theorem 7.2 (Equivariant witness DAG)**.** *A selected atlas row has a symbolic coloring recipe with at most one record for each reachable pair (grammar rule, selected row). The recipe realizes the row for* every *assignment of its visible component-root variables. Its size and its soundness verification are polynomial in the stored atlases and the grammar size. This bound is independent of the number of represented occurrences and of* 2^C*.*

Proof. Choose the stored realizing origin of each reached row. At a leaf, solve its actual cut-set equations. For a visible component, express the color of each vertex as X_i plus its normalized offset from the first terminal. For a hidden component, use a predecessor path in the four-state computation to choose one fixed root orientation producing the row's hidden mask. Its vertex colors are then constants. These choices realize the row for every visible input assignment.

At an internal record, choose its stored child rows and solve the signed quotient constraints. If a quotient component remains visible, its temporary root is the corresponding new root variable plus a fixed offset. If it becomes hidden, fix one orientation from a predecessor path realizing the specified parent hidden mask. A child visible root is therefore either one parent root variable plus an offset or a constant. Store these transport expressions alongside the child-row references.

By induction, substituting the transported variables into the child recipes yields compatible colorings of fresh child occurrences. The proof of Theorem 5.3 shows that the quotient coloring has the stated exact cost, relative tags and hidden mask. Importantly, the hidden orientations depend on the row but not on the parent visible inputs. Hence one recipe record suffices for the row, regardless of the root values requested by different ancestors.

Merge all repeated (rule,row) requests. A child's rule index is strictly smaller than its parent's, so this yields a DAG. Each leaf record has one expression per explicit vertex; each internal record has one transport expression per child visible component. These are polynomial-size records. An

independent soundness checker can verify leaf edge colors, terminal expressions, costs and tags, and at joins verify every quotient equality and transported tag expression. No enumeration of root assignments is needed. □

The symbolic checker in `checker.py` validates this argument directly. It rejects an expression using the wrong component variable even when an aggregate mask might coincidentally be constant. It verifies a positive coloring family without trusting any complete atlas. It does *not* prove that the selected cost is minimum or that another row is absent.

7.3 Original occurrence addresses and random access

An edge occurrence is addressed by a root-to-leaf sequence of child choices and a leaf edge index. Such a path distinguishes two fresh uses of the same rule. A vertex can likewise be queried at a leaf occurrence; synonymous vertex addresses created by quotient identifications agree because the recipe checker verified all symbolic quotient equalities.

Theorem 7.3 (Succinct cuts, marks and color access)**.** *For a selected row of cost $d \le k$, all cut-edge occurrence addresses can be extracted in total path length $O(ks)$. Given an actual visible-root assignment, one monochromatic-edge occurrence witnessing each present global color can also be extracted. There are at most two such addresses, each of length at most s. A requested leaf-occurrence vertex color is evaluated in polynomial time along its address without expanding sibling occurrences.*

Proof. Cost is invariant under all component root flips. Skip every cost-zero witness branch. A positive-cost internal record splits its cost into nonnegative child costs, and each positive child has integer cost at least one. Across any depth there are at most k active occurrence branches. The grammar depth is at most s, so at most ks positive-cost occurrence visits are needed. At a reached leaf, output exactly its stored cut edges. Their positive capacities sum to d, hence there are at most k of them.

For a requested monochromatic color, substitute the current actual root values into the record's tags. At a join, at least one child has the requested actual flag, because parent flags are their OR. Transport the root values to that child and continue. At a leaf, scan its explicit edges and return one with the requested endpoint color. There are at most s steps per mark. Finally, random vertex-color access is repeated substitution of the stored affine root expressions along a single address, followed by evaluation of one leaf expression. □

At threshold three, a positive terminal obstruction therefore carries at most three cut-edge addresses and two mark addresses, plus a checked symbolic recipe establishing global consistency. These are *grammar* addresses. Converting them into native triangulation charts or an embedded violating disc remains a separate geometric operation.

8 Application to terminal boundary-pattern tests

8.1 The geometric premises must remain explicit

A boundary-pattern application requires more than an abstract grammar. Assume that a source-faithful interpretation identifies its terminal-free root with a nonempty trivalent graph P embedded in the boundary sphere of a *certified 3-ball*. Assume also that the pattern has no vertex-free circle

components and that graph edges have unit capacity. The circle inventory and the ball certificate are external to the atlas.

For these inputs the two-edge-mark query is the graph form of the terminal obstruction test used in report 91 [5]. Lackenby's essential-pattern criterion gives its surrounding topological context [2, Proposition 2.6]. We recall the argument to make clear which part the present algorithm actually accelerates.

Lemma 8.1 (An edge forces a cycle on a small cubic cut shore)**.** *Let G be cubic, with loops contributing two to degree, and let $S \subseteq V(G)$ have $|\delta(S)| \leq 3$. If the induced graph $G[S]$ contains an edge, it contains a cycle.*

Proof. If $G[S]$ were a forest on v vertices with a nonempty components, it would have $v - a$ edges. The degree sum would give

$$|\delta(S)| = 3v - 2(v - a) = v + 2a.$$

A forest containing an edge has $v \geq 2$ and $a \geq 1$, so this is at least four, a contradiction. Loops and double-edge cycles are already cycles and cannot occur in a forest. □

Proposition 8.2 (Terminal graph interpretation, recalled)**.** *Under the stated geometric premises, P is inessential on the ball if and only if its abstract graph has a coloring of cut cost at most three with both monochromatic-edge marks.*

Proof. A transverse simple closed curve on the sphere meets the pattern in at most three points. If the pattern in one of its complementary discs is a forest, let v be its interior trivalent vertices, b its boundary endpoints, and a its nonempty tree components. The tree degree sum gives $b = v + 2a$. At $b \leq 3$, the forest is therefore empty, one arc, or one tripod. The exclusion of vertex-free circle components ensures that these alternatives account for the acyclic cases. Consequently, a violating curve has a cycle of P on each side. Coloring original graph vertices by side cuts at most the intersected edges, and both cycles supply monochromatic-edge marks.

Conversely, a coloring with both marks and cut size at most three has a cycle in each induced shore by Lemma 8.1. Suppose first that the graph is connected. Choose a smallest edge cut with a cycle on both shores. Both shores can be taken connected: otherwise retain a component containing a cycle on one shore and move the other components to the opposite shore. Its boundary is no larger, and is strictly smaller when any extra component was present, since the ambient graph is connected. The opposite cycle is retained. Apply the same reasoning to the other shore. Such a cut is a bond. In a sphere embedding, a bond is represented by a dual simple cycle, hence by a transverse simple closed curve meeting exactly its cut edges. Both sides contain cycles, so neither side is an allowed empty/arc/tripod disc.

If the graph is disconnected, every nonempty cubic component contains a cycle. With at least two components there is a separating simple closed curve disjoint from the graph, with a component on each side. It is already a zero-intersection violating curve. The connected case and this disconnected case cover all possibilities. Every sphere curve bounds a properly embedded disc in the ball, so the curve interpretation agrees with the pattern-disc test. □

Corollary 8.3 (Polynomial supplied-grammar terminal test)**.** *For a source-authenticated grammar satisfying the preceding geometric premises, the terminal essential-pattern query is polynomial in the encoded grammar size at threshold three, without any connectedness or component-bound condition on intermediate rules.*

Proof. Compute the closed-root atlas by Theorem 6.3. It contains a row with hidden mask 3 exactly when the obstruction of Proposition 8.2 exists. Use complete replay for the negative answer and a symbolic coloring recipe for a positive answer. □

The empty pattern and a single vertex-free circle are separate essential cases. Multiple vertex-free circles or a circle mixed with a nonempty graph require an explicit circle-component inventory and their corresponding topological handling; they are not invented as graph vertices. Likewise a sphere boundary by itself does not authenticate a ball. The command-line program consequently returns only `MARKED_CUT`, `NO_MARKED_CUT`, or an open/inconclusive status, never `UNKNOT`.

8.2 What changes in the global complexity ledger

The graph theorem removes a particular condition from the predecessor's pipeline: intermediate pieces no longer need to remain connected or have logarithmically many terminal-bearing components. It does not bound the number of emitted geometric grammar rules, native interface vertices, decomposing-surface choices, or hierarchy restarts.

For clarity, suppose the non-graph part of an algorithm takes time $T_{\mathrm{geom}}(n)$ and issues graph queries $\mathcal{G}_1, \dots, \mathcal{G}_m$ with thresholds k_j. The present result yields the honest transfer estimate

$$T_{\mathrm{total}}(n) \le T_{\mathrm{geom}}(n) + \sum_{j=1}^{m} \mathrm{poly}(s_j, L_j, W_j, B_j, k_j)(L_j + W_j + 2)^{O(k_j)}. \tag{13}$$

If all $k_j = 3$, the graph part is polynomial in the total listed graph-grammar data, up to a fixed polynomial degree. Therefore a construction producing at most $2^{O((\log n)^3)}$ such data and spending at most that much other time would retain Lackenby's announced asymptotic scale after replacing this operator. Equation (13) is a conditional transfer statement, not a proof that the required geometric bounds hold.

Report 92's causal-restart accounting [7] concerns a different source of complexity: when a repair invalidates later geometric constructions, a cheap local graph query does not by itself prevent the suffix from being rebuilt. Distinguishing this causal depth from the arbitrary *graph grammar* depth allowed by our theorem avoids a potentially serious conflation.

9 Implementation, source compatibility, and trust boundaries

9.1 Delivered modules and certificate contracts

The package is standard-library Python, with no dependency on a graph library, a numerical optimizer, or the repository's native topology engine. The producer is `src/gauge_cuts.py`; the independent checker is `src/checker.py`. The checker imports neither the producer nor its parity disjoint-set structure. Leaf production uses forest switches; leaf replay uses exact edge-cut sets. Internal replay uses signed-graph traversal. The independent explicit oracle in `src/reference.py` enumerates literal vertex colorings on capped small graphs.

`src/coloring.py` produces equivariant recipes and implements random-access colors and monochromatic-edge address extraction. The complete packet schema is `component-gauge-cut-atlas-v1`; the positive symbolic schema is `component-gauge-coloring-v1`. They are intentionally distinct. The complete checker recomputes the root query; packet statistics are performance annotations, not premises of a proof.

A row is serialized as

```
[exact_cost, terminal_parity_integer, component_tag_list, hidden_mask]
```

A terminal parity integer has one bit per ordered terminal. A leaf origin lists cut-edge indices, and an internal origin names one child row per child occurrence. A symbolic vertex expression is `[i,b]` for root variable $X_i + b$ or `[-1,b]` for a constant. Quotient namespaces are dense explicit integer lists. The validators reject duplicate terminal declarations, forward/cyclic references, invalid capacities, and unsupported operations.

9.2 A tested adapter to the existing report-91 input schema

The source contract in report 91's `src/sparse_cuts.py` was inspected at the pinned revision; its blob is listed in Appendix A. The additive adapter `src/report91_adapter.py` accepts its `proveit.graph-grammar.v1` schema, translates leaves and quotient rules, and preserves the selected root. A selected nonfinal root becomes the child of an appended identity map; unused source rules do not change its graph.

Proposition 9.1 (Adapter semantics)**.** *Every accepted source grammar and its translated grammar have isomorphic expanded selected-root multigraphs, preserving edge occurrences, capacities, and ordered terminals. Translation has polynomial size in the listed source data.*

Proof. At a leaf the adapter only changes the representation of the vertex inventory. Before creating the explicit list it checks the source's terminal/edge/private-isolate coverage condition; hence an arbitrarily large unlisted count is rejected. Internal rules use the same child references, quotient labels, and retained terminal order. Induction over rules identifies the fresh-copy expansions. If needed, the final identity map preserves the selected graph and interface. Each translated vertex is supported by an explicit source inventory entry, apart from the source schema's allowed single isolated vertex, so the size increase is polynomial. □

The actual report-91 triangular-prism fixture was read through the GitHub connector and reproduced as `examples/report91_triangular_prism.json`; formatting was normalized, not its JSON graph data. Its four-rule grammar was translated, compiled, checked, and compared at every rule with a literal coloring oracle. The root has six vertices, nine edges, is cubic, and has minimum both-mark cut cost three. The resulting four tables contain 53 rows; full replay considers 130 leaf/rule candidates. The recorded result is `data/report91_adapter_result.json`.

This is a real compatibility check against an existing repository graph fixture. It is not a rerun of report 91's entire software suite, a test of the native triangulation pipeline, or a source-geometric importer. No production module was overwritten and no repository write was performed.

9.3 Operational limitations that matter

A work budget aborts with an inconclusive result. It must not be converted into absence of a cut. The programmatic integer API was also tested with a capacity of 2^{16384}; typed structural source comparison avoids converting that value into a long decimal string. Python's ordinary JSON text parser/encoder has its own configurable large-decimal-integer limit, so that particular stress input is generated programmatically rather than advertised as an unrestricted default-JSON round trip.

The package implements exact-cost atlases and symbolic witness export. It does not yet implement the precompiled linear gluing plan of Proposition 5.5, the minimum-only mode of Proposition 6.5, or

an automatic production hybrid. Their proofs and explicit scope are retained as implementable next steps, not benchmarked features.

10 Executed validation and paired measurements

10.1 Finite validation

The recorded run uses Python 3.13.5 and a fixed audit seed 20261012. There are 48 focused unit-test methods. The main audit additionally performs the checks in Table 1. Counts refer to this delivery, not to tests inherited from earlier reports.

Completed validation task	Count
Labeled simple graphs, zero through five vertices	1,100
Graph/threshold comparisons	4,400
All terminal-subset projections	33,867
Weighted multigraph leaf comparisons	1,000
Random fresh-copy grammars	1,000
Literal expanded grammar-node comparisons	8,796
Complete certificate replays	3,105
Symbolic coloring and cut-address replays	2,102
Cut-edge occurrence addresses checked	205
Atlas size / shared-budget bound checks	12,056 / 4,467
Forged metadata packets rejected	1,000

Table 1: Executed deterministic audit. The 48 unit-test methods and the separate report-91 adapter checks are additional. Counts are validation tasks, not independent proofs.

All labeled simple graphs on zero through five vertices are included, both connected and disconnected. For each graph the all-terminal atlas is compared with literal colorings; thresholds zero through three are checked, and all terminal subsets are obtained by the proposed forgetting operation and compared with a fresh literal oracle. The 33,867 projection cases are therefore checks of both normalization and elimination, not just counts of input graphs.

The weighted-leaf audit uses up to eight explicit vertices, loops and parallel edges, shuffled terminal orders, and positive capacities from one through seven. The random-grammar generator permits unary self-identifications, repeated child references, empty interfaces, terminal reordering, and forgetting. It caps literal expansions at ten vertices and 45 edges while generating rules, skipping a proposed rule that would exceed those caps. It does not skip a grammar because of an incorrect result. These are finite tests, not a statistical proof of the arbitrary-size theorem.

Full certificate replay is also tested against altered metadata. Focused tests additionally corrupt costs, flags, origins, source bindings, root answers, leaf symbolic colors and internal symbolic transports. The symbolic-witness checks include all 256 assignments of eight independent visible root bits, checking random-access colors without changing the stored recipe.

10.2 Binary stress cases

Five deliberately different large-encoding cases were executed. A hidden disjoint-union power has 2^{512} edge components but only 513 grammar rules. A serial power of two unit-capacity permuted channels uses 128 squarings. A second serial test uses 32 channels, capacities four, and 512 squarings.

A fourth case has 1,024 isolated terminal components in one explicit leaf. The last uses the 16,385-bit capacity described above.

Supplied graph description	Rules	Peak rows	Total rows
Hidden edge components, 2^{512}	513	12	6,140
Unit-capacity channels, 2^{128} copies	130	39	4,978
32 heavy channels, 2^{512} copies	514	3	516
1,024 isolated terminal components	1	1	1
One 16,385-bit edge capacity	2	2	3

Table 2: Binary stress cases, all with complete independent replay. Row counts measure the atlas, not the number of expanded graph vertices or components.

The channel examples are abstract graph stress cases. Closing even powers in the chosen fixture yields disconnected cycles, so the both-mark query can have cost zero by coloring different cycles differently. Their role is to test nested fresh-copy composition, component gauges, exact flags and source addresses; they are not difficult knot diagrams. Full certificates for the four JSON-friendly cases are included, together with a symbolic coloring recipe for the unit-channel power.

10.3 Paired complete-query protocol

The explicit-root baseline is an independently written response-table evaluator for the same grammar operations. It stores exact costs, actual terminal colorings and the global flag mask. The tested output is the same serialized *complete exact terminal-free response*, not a single favorable selected row. Both paths evaluate all supplied rules. The gauge path also prepares realizing origins. Complete certificate verification is measured separately; it is not silently included in one side of the paired timings.

Each case has an untimed agreement check and five paired repetitions, alternating method order with garbage collection before each timing. Table 3 reports medians in milliseconds. The raw rows are in `data/benchmark_raw.csv`; environment and complete summaries are in `data/benchmark.json`. Five gauge-versus-gauge control pairs have second/first ratios from 0.862 to 1.099, illustrating local timing variability.

Family / parameter	Gauge ms	Explicit ms	E/G	Gauge rows	Explicit rows
Components $c = 1$	0.164	0.089	0.55	2	2
Components $c = 2$	0.194	0.107	0.55	2	4
Components $c = 4$	0.247	0.156	0.63	2	16
Components $c = 8$	0.369	1.232	3.34	2	256
Components $c = 12$	0.501	23.091	46.06	2	4,096
Components $c = 16$	0.741	503.127	679.11	2	65,536
Components $c = 18$	0.775	2,232.279	2,879.44	2	262,144
Power depth $h = 1$	0.469	0.460	0.98	15	54
Power depth $h = 4$	8.309	7.138	0.86	39	66
Power depth $h = 16$	47.461	36.587	0.77	39	66
Power depth $h = 64$	196.604	153.063	0.78	39	66

Table 3: Paired complete graph-query medians, five repetitions. E/G is explicit time divided by gauge time; below one favors the explicit backend. The last two columns are peak stored states over all rules. The second family has two unit-capacity channels.

The independent-component family builds c heavy edges with separate visible roots, then identifies all roots and forgets them. Its supplied parse has an intermediate 2^c explicit boundary table while the gauge representation has one intermediate row and two closed-root rows. The large gains on this family substantiate removal of the explicit component factor. A different parse of the final star would be easy for an ordinary connected-piece algorithm; we do not claim a representation-independent exponential lower bound.

The permutation-power family is an important negative control. At two channels, the atlas reduces candidates but incurs more Python-level parity and per-component-tag work. At depth 64 it takes about 196.6 ms versus 153.1 ms for the explicit-state baseline. Thus this implementation is not a uniformly faster backend. A production integration should preserve the simpler backend for favorable small interfaces and select the gauge backend when explicit component states threaten to dominate. That dispatch recommendation is based on these graph benchmarks, not measured native recognition speed.

11 Further research questions and proposed next steps

11.1 A faithful native boundary-grammar exporter

The most valuable missing theorem is not another bound on the number of free component bits. It is an encoded-size theorem for converting the native marked parallelity/core boundary data into the permitted graph grammar while preserving every original edge occurrence, actual corner identification, vertex-free circle component, and cyclic embedding datum. Which native interval families admit polynomial-size graph rules without enumerating ranks? A satisfactory exporter should produce both the graph recipe and a separately checked source chart interpretation. The report-91 schema adapter solves neither part of this geometric problem.

11.2 Compile signed interfaces once

Implement Proposition 5.5 with packed bit operations and compare it against parity disjoint sets and the explicit-root backend. The metadata partition fixes the cycle checks; repeated candidate pairs should not rebuild its topology. A useful target is a certificate format containing the precompiled plan and a small checker that verifies it against the quotient maps. This is particularly relevant to the measured small-width regressions.

11.3 A minimum-only certificate mode and honest hybrid dispatch

Implement Proposition 6.5 under a new schema and independently check completeness of the minimum response rather than of exact cost support. Determine whether min-only gauges beat the existing minimum-state algorithm on narrow connected and disconnected interfaces. A hybrid should use measurable work estimates or resource limits, preserve all certificate semantics, and report inconclusive outcomes rather than silently truncating states. Dispatch must not become part of the soundness argument.

11.4 From graph addresses to violating discs

Theorem 7.3 provides a succinct actual coloring, up to three cut edges and two monochromatic-edge witnesses for the graph obstruction. The remaining geometric task is to reconstruct a bond/noose in

the authenticated spherical embedding and transport its pushed-in disc through the hierarchy using original source charts. Can this be done with polynomial encoded size independently of the number of repeated pattern edges? Tracking only graph-edge indices is not enough when a source edge has several geometrically distinct rank occurrences.

11.5 Dynamic repairs and honest causal dependencies

An edit to one source rule requires rebuilding only its graph-DAG ancestors, with exact cost and flags recomputed by the same theorem. This easy graph-local statement should be separated from a theorem that a *geometric* hierarchy repair preserves unrelated rules or source interpretations. Which compressed chart invariants certify such persistence? Connecting a verified native exporter to the source-conditional causal framework of report 92 is a substantive next research direction, rather than assuming that an unchanged graph table authenticates an unchanged manifold piece.

11.6 Sharper atlas bounds and extended observations

The constants in Theorem 4.2 are deliberately loose. Can one characterize the attainable component tags for each relative terminal pattern and exact small cost to reduce the degree-three constant? The star lower bound prevents eliminating the r^k scale in an explicit normalized-pattern atlas, but does not rule out additional algebraic sharing between those rows.

A broader question replaces the two edge flags by a fixed finite observation algebra acted on by color complement. Closure under gluing still follows when observations compose equivariantly and hidden elimination has a fixed finite state space. A polynomial row theorem additionally needs a unique zero-cost normalized observation per visible component, or another controlled baseline. Pinning many internal absolute colors or admitting zero-cost edges can destroy the present counting argument; extensions must restate their hypotheses rather than borrow the bound unchanged.

11.7 Higher thresholds and genuine global progress

For thresholds of order $\log n$, the local operator is already quasi-polynomial under polynomial grammar bounds. At threshold three it is polynomial. Therefore obtaining the global scale (2) now rests on geometric encoding and restart accounting, not on this component-color enumeration. A useful next milestone is a source-verified family of nontrivial hierarchy repairs whose complete grammar production, graph queries, symbolic witness transport, and restart counts all satisfy one common encoded-size recurrence.

12 Claims ledger and conclusion

Claim	Status in this delivery
Component-gauge semantics and exact composition	Proved for the specified positive-integral, edge-owned, terminal-only graph model.
Polynomial fixed-threshold evaluation without a component/depth bound	Proved, including bit-size, shared-budget and leaf-inventory accounting.
Equivariant coloring recipes and occurrence addresses	Proved, implemented, and independently soundness-checked.

Claim	Status in this delivery
Complete atlas absence/minimality checking	Implemented by a separate algorithm and exercised in the recorded audit.
Precompiled parity plans and min-only dominance	Proved as optimization propositions; not implemented or timed here.
Report-91 graph-schema compatibility	Implemented and tested on the actual pinned triangular-prism fixture and edge cases.
Native geometric grammar exporter and disc transport	Not implemented or established by this contribution.
Performance improvement	Measured on the stated abstract graph families; includes both large gains and regressions.
Unrestricted quasi-polynomial unknot recognition	Not claimed by this delivery; geometric production and global accounting remain additional requirements.
Literature priority	No exhaustive novelty-priority claim. The predecessor question and classical ingredients are explicitly identified.

The essential mathematical change is to stop treating independent component root colors as separate states. Relative terminal parities alone would be insufficient: the monochromatic-edge evidence must remain attached to the same component variables. Once this information is retained, gluing is a consistency problem over $\mathbb{F}_2$, and forgotten roots are eliminated through four possible aggregate masks. Positive capacity then confines nonbaseline behavior to a small total cut budget. This combination gives a closed, polynomial fixed-threshold operator for arbitrary supplied disconnected graph grammars and retains polynomial-size witnesses. It is a concrete removal of one previously identified asymptotic obstruction in the ProveIt hierarchy program.

A Source audit and attribution

The repository snapshot used for the compatibility and maintained-source checks is

`d5c9a97fac1803258ea675297e4a7b22b78dd79b`

Its branch metadata was fetched during this research session. This is a targeted source audit, not a complete repository review or a claim that every incoming archive was downloaded. The initial incoming directory listing was inspected; the later report index at the pinned commit shows the relevant deliveries placed in reports 88–97.

Inspected object	Scope and identifier
Maintained parallelity synthesis	`synthesis/parallelity.tex`, lines 1–160; blob `d7293976bea14fce6b4848bf6a2d7a20134c2491`. Confirms all-dimensional source bundle and $30T$ inherited-core statement.
Maintained primitive-annulus synthesis	`synthesis/bundle_primitive_annuli.tex`; blob `79123c0b72afaa74d741443a16542b0d25ffc35d`. Inspected as existing progress, not re-proved or integrated here.

Inspected object	Scope and identifier
Report index	`reports/README.md`, relevant placement/status lines; pinned blob `f174a162be60bbab40eef4023ba1e345f20e8b98`.
Sparse-cut predecessor article	Library source `Sparse_Cut_Grammars.tex`, including definitions, proofs, complexity, geometric premises, questions and bibliography. Its component-factor question is the research target.
Report-91 input contract	`reports/91/src/sparse_cuts.py`, lines 1–212; blob `5ee84ac65474d315f50f0b1640094ec7a6da3450`. Used to implement the additive adapter.
Report-91 prism fixture	`reports/91/examples/triangular_prism.json`; blob `9425102ac5e5c63158e8f5d38e0e97fa8505f9e1`. Graph data reproduced with normalized JSON formatting and tested.
Other precursor material	Targeted Library abstracts/sections for disconnected tile powers, moment-coded first returns, modular Morse certificates, and cographic persistence; not a full audit of those software packages.
Lackenby announcement	Official Oxford March 2021 handout; pages 7 and 25 visually checked for the precise exponent.
Lackenby preprint	arXiv:2607.23350v1, 25 July 2026; introduction, Proposition 2.6, Section 9 and relevant hierarchy discussion checked.

The package does not contain copies of external papers or presentation slides. No external full article is redistributed. The code in this delivery is new standalone research code; its graph-schema adapter records the source contract it follows.

B Reproduction and integration

From the delivery root:

```
PYTHONPATH=src python -m unittest discover -s tests -v
PYTHONPATH=src python experiments/replay_delivery.py
PYTHONPATH=src python experiments/audit.py
PYTHONPATH=src python experiments/benchmark.py
PYTHONPATH=src python src/query.py examples/prism.json \
    --check --certificate /tmp/prism-atlas.json
PYTHONPATH=src python integration/query_report91.py \
    examples/report91_triangular_prism.json
cd article && sh build.sh
```

The standard-library programs target Python 3.10 or later; the recorded run used Python 3.13.5. Rebuilding the PDF requires `pdflatex` and the packages named in this source. The TeX document is standalone: its displayed data tables are embedded rather than fetched over a network. Re-running measurements changes timing output, not mathematical content.

For symbolic witness use:

```
from gauge_cuts import compile_grammar
from checker import verify_packet, verify_coloring_recipe
from coloring import build_recipe, mark_address
packet = compile_grammar(grammar)
verify_packet(packet, expected_grammar=grammar)
ri = next(i for i,r in enumerate(packet["atlases"][-1]["rows"])
          if r[3] == 3)
recipe = build_recipe(packet, ri)
verify_coloring_recipe(recipe, expected_grammar=grammar)
marks = [mark_address(recipe, b) for b in (0, 1)]
```

This example presupposes a terminal-free root with a both-mark row. Production callers must branch on the query result rather than treating failure of `next` as a topological verdict.

The recommended intake is an intact new report directory under `Topology/UnknotRecogniti on/reports/`, with its number chosen at integration time. The additive adapter may be reviewed separately from native integration. `integration/INTEGRATION.md` identifies the operator interface, input premises, proposed future dispatch, and explicit exclusions. The package includes tests, raw timings, complete graph certificates, symbolic coloring recipes, source-audit metadata and a concise claims file. Generated caches and LaTeX build intermediates are not part of the delivery.

References

[1] M. Lackenby, *Unknot recognition in quasi-polynomial time*, Oxford topology seminar handout, March 2021, especially slides 7 and 25. https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk-oxford-compressed.pdf.

[2] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. https://arxiv.org/abs/2607.23350.

[3] ProveIt contributors, *Source parallelity in all handle dimensions and a bounded inherited core*, maintained synthesis, 2026. Source revision and inspected range in Appendix A. https://github.com/VladimirReshetnikov/ProveIt/blob/d5c9a97fac1803258ea675297e4a7b22b78dd79b/Topology/UnknotRecognition/synthesis/parallelity.tex.

[4] ProveIt contributors, *Unknot-recognition reports index*, inspected at the pinned revision, 12 October 2026. https://github.com/VladimirReshetnikov/ProveIt/blob/d5c9a97fac1803258ea675297e4a7b22b78dd79b/Topology/UnknotRecognition/reports/README.md.

[5] ProveIt research continuation prepared with ChatGPT, *Sparse Small-Cut Responses for Compressed Unknot Hierarchies: Connected graph grammars, budget-preserving composition, and source-addressable terminal certificates*, 12 October 2026. Library manuscript `Sparse_Cut_Grammars.tex`; delivery placed as report 91. https://github.com/VladimirReshetnikov/ProveIt/tree/d5c9a97fac1803258ea675297e4a7b22b78dd79b/Topology/UnknotRecognition/reports/91.

[6] ProveIt research continuation, *Polynomial Marked-Cut Kernels for Arbitrary Disconnected Tile Powers*, 12 October 2026. Delivery placed as report 88. https://github.com/VladimirReshetnikov/ProveIt/tree/d5c9a97fac1803258ea675297e4a7b22b78dd79b/Topology/UnknotRecognition/reports/88.

[7] ProveIt research continuation, *Cographic Persistence and Causal Reset Complexity*, 12 October 2026. Delivery placed as report 92. https://github.com/VladimirReshetnikov/ProveIt/tree/d5c9a97fac1803258ea675297e4a7b22b78dd79b/Topology/UnknotRecognition/reports/92.

[8] P. Chalermsook, S. Das, B. Laekhanukit, Y. Kook, Y. P. Liu, R. Peng, M. Sellke, and D. Vaz, *Vertex Sparsification for Edge Connectivity*, SODA 2021, 1206–1225; arXiv:2007.07862. https://arxiv.org/abs/2007.07862.

[9] Y. P. Liu, *Vertex Sparsification for Edge Connectivity in Polynomial Time*, ITCS 2023, LIPIcs 251, 83:1–83:15. https://doi.org/10.4230/LIPIcs.ITCS.2023.83.